```
python
```

```
"""
game_draw_unified.py — Drop-in replacement for game_core.game_draw()

This module provides the unified game_draw() function that replaces
the mission-routed dispatcher in game_core.py. Instead of:

    def game_draw(fcx, fcy):
        if _is_mission2_active():
            return mission2.game_draw(fcx, fcy)
        if _is_mission3_active():
            return mission3.game_draw(fcx, fcy)
        return mission.game_draw(fcx, fcy)

You now call:

    from game_draw_unified import game_draw
    game_draw(fcx, fcy)

and it routes to the correct GameRenderer population method.

Integration steps:
  1. Copy render_pipeline.py, game_renderer.py, and game_draw_unified.py
     into your game's directory (alongside config.py, state.py, etc.)

  2. In mainloop.py (or wherever game_draw is called), change:
         gc.game_draw(cam_fcx, cam_fcy)
     to:
         from game_draw_unified import game_draw
         game_draw(cam_fcx, cam_fcy)

  3. For each mission, fill in the placeholder draw methods in
     game_renderer.py with the actual code from mission.py / mission2.py /
     mission3.py. Each placeholder method has a "Replace with..." comment.

  4. Optionally remove the old mission-specific game_draw() functions
     once all drawing code has been migrated to the renderer.

Benefits:
  - Adding a new visual layer = adding one pipeline.submit() call
  - Y-sorting entities is automatic (sort_key=sort_centery)
  - Post-processing is always last, always consistent
  - No more duplicated draw code between missions
  - New missions just add a populate_mX() method
"""

def game_draw(fcx, fcy):
    """Unified game draw entry point.

    Routes to the correct GameRenderer population method based on
    the active mission, then flushes the pipeline.
    """
    from game_renderer import GameRenderer
    import game_core as gc

    active = getattr(gc, "active_mission", "MISSION1")

    renderer = GameRenderer(fcx, fcy, [illegible])

    if active == "MISSION2":
        renderer.populate_m2()
    elif active == "MISSION3":
        renderer.populate_m3()
    else:
        renderer.populate_m1()

    renderer.flush()
```